

# Deep Dive: find, Pipelines & Worked Examples

Everything up to Challenge A, solved together — Challenge B and C stay yours.

```
analyst@kali: ~  
$ echo "small tools, connected, beat one big tool"  
small tools, connected, beat one big tool
```

**Stefanus Eko Prasetyo**

Lab-1 · companion to the Lab 01 lecture

# What we cover today

## 1 find in depth

combining conditions with -o, time filters (-mmin vs -mtime), exact dates with -newermt

## 2 How pipelines work

what | really does, and the classic forensic pipelines, stage by stage

## 3 The forensic option pack

the options that matter for every command in this lab — ls, cat, strings, grep, find, hashing, xxd, hexedit, ps, netstat

## 4 Worked examples

the guided drills 3.A-3.E and Challenge A, fully solved on screen

## 5 Your turn

Challenge B (the lying header) and Challenge C (the log hunt) — no spoilers here

# Combining conditions — AND by default, -o for OR

```
analyst@kali: ~  
$ find files/images -name "*.jpg" -o -name "*.png"  
files/images/hero.jpg  
files/images/logo.png  
  
# -o = OR: show files matching EITHER condition  
  
$ find . -name "*.jpg" -mtime -30  
# no -o -> conditions are AND-ed:  
#   a .jpg that ALSO changed in the last 30 days  
  
# THE GOTCHA: -o binds looser than actions.  
$ find . -name "*.jpg" -o -name "*.png" -print  
# ^ prints ONLY the png branch! group instead:  
$ find . \( -name "*.jpg" -o -name "*.png" \) -print
```

## Rules of thumb

- ▶ multiple conditions with no -o are silently AND-ed
- ▶ -o shows a file if ANY branch matches
- ▶ the moment you add -print, -exec, or -delete: wrap the OR in \( \)
- ▶ test a complex find with -print BEFORE wiring in -delete

# Time filters — -mmin in minutes, -mtime in 24-hour blocks

```
analyst@kali: ~  
  
$ find /home -mmin -30  
/home/analyst/notes.txt  
# changed LESS than 30 minutes ago  
  
$ find /var/log -mtime -1  
# changed within the last 24 hours  
  
$ find /case -mmin +60  
# last change OLDER than 60 minutes ago  
  
# -mtime counts FULL 24h blocks, fractions dropped:  
#   -mtime 0  = 0..24 h ago  
#   -mtime 1  = 24..48 h ago  (not "yesterday!")  
#   -mtime +7 = older than 8 x 24 h
```

## Signs and units

- ▶ -n = within the last n units (younger)
- ▶ n = exactly the n-th window back
- ▶ +n = older than n units
- ▶ minutes precision needs -mmin: 15 days = -mmin -21600
- ▶ relatives: -amin/-atime (access), -cmin/-ctime (metadata)

# Exact dates — -newermt and a range

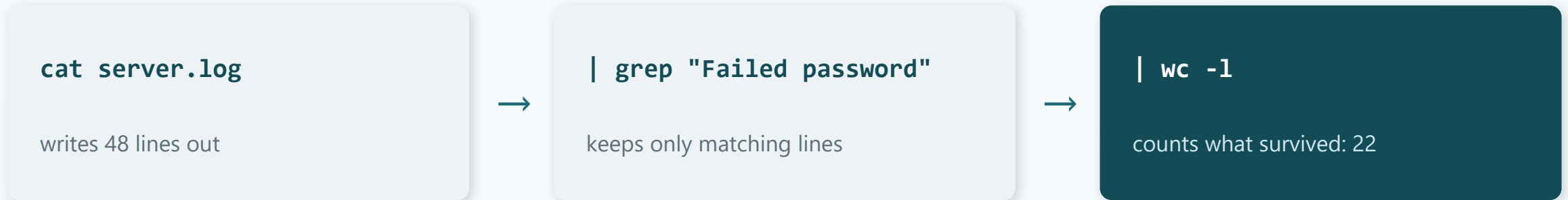
```
analyst@kali: ~  
  
# modified ON 15 September 2026 (whole day):  
$ find / -newermt "2026-09-15 00:00:00" \  
>      ! -newermt "2026-09-16 00:00:00"  
  
# modified between 1 and 15 September:  
$ find /case -newermt "2026-09-01" ! -newermt "2026-09-16"  
  
# files NEWER than an evidence file you seized:  
$ find / -newer seized_usb.dd
```

## Reading it right

- ▶ -newermt "date" = mtime on/after that moment
- ▶ ! -newermt "date" = mtime at or before it
- ▶ together they bracket an exact window
- ▶ careful: filesystem times can be forged (timestomping) — the serious timeline comes from \$MFT in Lab 06

# What the | really does

The pipe connects two commands: everything the left command writes to standard output becomes the standard input of the right one. Each tool stays small — the power is in the connection.



```
analyst@kali: ~  
$ cat server.log | grep "Failed password" | wc -l  
22  
# same result as grep -c — the pipe just makes every tool reusable
```

# Four pipelines you will use every week



analyst@kali: ~

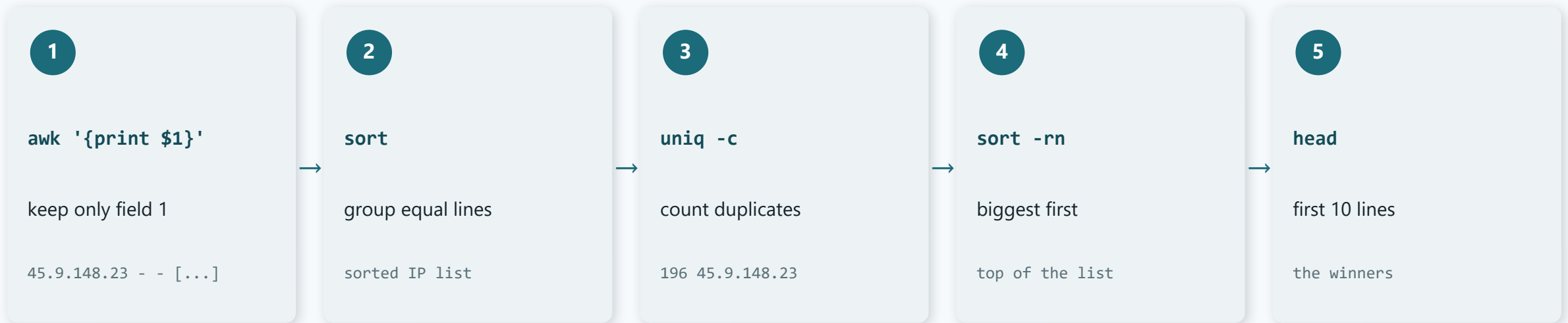
```
# 1. count lines that matter
$ grep "Failed password" server.log | wc -l

# 2. rank client IPs by request count
$ awk '{print $1}' access_big.log | sort | uniq -c | sort -rn | head
#    196 45.9.148.23      <- there is our attacker
#      8127 10.0.1.4      (top normal client for contrast)

# 3. readable text out of a binary, filtered
$ strings -n 6 update.bin | grep -E "http|config"

# 4. how many images did we collect?
$ find files/images \( -name "*.jpg" -o -name "*.png" \) | wc -l
```

# Anatomy — ranking IPs, stage by stage



```
$ awk '{print $1}' access_big.log | sort | uniq -c | sort -rn | head -3
196 45.9.148.23
139 198.51.100.9
4127 10.0.1.4
```

analyst@kali: ~

# Listing & identifying — ls, cd, file

| Option                          | What it does                       | Forensic value                              |
|---------------------------------|------------------------------------|---------------------------------------------|
| <code>ls --full-time</code>     | timestamps with seconds + timezone | exact ordering of events                    |
| <code>ls -a</code>              | include hidden dotfiles            | malware and configs love to hide here       |
| <code>ls -i</code>              | show inode numbers                 | correlate with find -inum and \$MFT records |
| <code>ls -lt / -lu / -lc</code> | sort by mtime / atime / ctime      | freshhest artifacts first; spot odd edits   |
| <code>ls -R</code>              | recursive listing                  | inventory a whole tree fast                 |
| <code>cd -</code>               | jump to the previous directory     | quick bounce between evidence and copy      |
| <code>file -b</code>            | brief output, no filename          | clean, scriptable pipeline input            |
| <code>file -z</code>            | peek inside compressed files       | archives wearing disguises                  |
| <code>file -k</code>            | keep checking, show all matches    | catches layered tricks                      |

# Reading content — cat, head/tail, strings

| Option                       | What it does                                  | Forensic value                                 |
|------------------------------|-----------------------------------------------|------------------------------------------------|
| <code>cat -n</code>          | number every line                             | cite evidence by exact line number             |
| <code>cat -A</code>          | reveal \$ line-ends, ^I tabs, trailing spaces | catch hidden whitespace (the evidence_B trick) |
| <code>head -n 20</code>      | first 20 lines only                           | sample a huge file safely                      |
| <code>tail -n 20 / -f</code> | last lines / follow live growth               | end of a log; watch an incident unfold         |
| <code>strings -n 8</code>    | minimum string length 8                       | drops noise; default is only 4                 |
| <code>strings -t x</code>    | print the hex offset of each string           | locate exactly where it sits in the file       |
| <code>strings -e l</code>    | decode UTF-16LE (wide chars)                  | Windows artifacts store wide strings           |

# grep — the workhorse

| Option                | What it does                      | Forensic value                              |
|-----------------------|-----------------------------------|---------------------------------------------|
| grep -i               | case-insensitive match            | attackers mix case to evade filters         |
| grep -c / -n          | count matches / show line numbers | size the problem; pinpoint evidence         |
| grep -r               | search directories recursively    | sweep whole trees without find              |
| grep -E               | extended regex                    | one pattern, many words: sqlmap nikto zgrab |
| grep -o               | print only the matching part      | extract clean IPs and URLs                  |
| grep -v               | invert the match                  | subtract known-good noise                   |
| grep -A 3 -B 3 (-C 3) | context lines after/before/around | the events surrounding an incident          |

analyst@kali: ~

```
$ grep -oE "[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+" access_big.log | sort -u | head -3
10.0.1.4    185.220.101.4    45.9.148.23
```

# find — beyond time filters

| Option                          | What it does                       | Forensic value                                                       |
|---------------------------------|------------------------------------|----------------------------------------------------------------------|
| <code>-type f / d / l</code>    | regular file / directory / symlink | separate real files from links                                       |
| <code>-size +10M</code>         | larger than 10 MB                  | hunt droppers, images, memory dumps                                  |
| <code>-user root</code>         | owned by a specific user           | who put it there?                                                    |
| <code>-perm -4000</code>        | setuid bit set                     | privilege-escalation inventory                                       |
| <code>-inum 1042</code>         | by inode number                    | same data under different names (hard links)                         |
| <code>-maxdepth 2</code>        | limit descent depth                | keep sweeps fast and targeted                                        |
| <code>-exec md5sum {} \;</code> | run a command on each hit          | hash everything you find in one line                                 |
| <code>\( -o \)</code>           | group OR conditions                | mandatory before <code>-exec</code> / <code>-delete</code> (slide 3) |

# Hashing — the verify loop

| Option                                   | What it does                         | Forensic value                  |
|------------------------------------------|--------------------------------------|---------------------------------|
| <code>md5sum file &gt; hashes.txt</code> | record hashes at acquisition         | your integrity baseline         |
| <code>md5sum -c hashes.txt</code>        | re-check every file against the list | OK or FAILED, per file          |
| <code>sha1sum -c</code>                  | same workflow with SHA1              | stronger than MD5               |
| <code>sha256sum</code>                   | SHA-256 digest                       | the professional standard today |

analyst@kali: ~

```
$ sha1sum evidence.dd copy.dd > hashes.txt
$ sha1sum -c hashes.txt
evidence.dd: OK
copy.dd: FAILED
# run -c once at acquisition, once before you report. FAILED = stop and investigate
```

# Hex toolkit — xxd & hexedit

| Option                         | What it does                     | Forensic value                   |
|--------------------------------|----------------------------------|----------------------------------|
| <code>xxd -l 32</code>         | dump only the first 32 bytes     | quick signature check            |
| <code>xxd -s 12</code>         | start at byte offset 12          | read chunk headers like IHDR     |
| <code>xxd -g 1</code>          | one byte per group               | byte-exact comparison            |
| <code>xxd -r</code>            | reverse: hex text back to binary | rebuild a patched file from text |
| <code>hexedit: Tab</code>      | toggle hex/ASCII column          | patch text bytes directly        |
| <code>hexedit: / and F2</code> | search; save (Ctrl+C quits)      | navigate, fix, verify            |

analyst@kali: ~

```
$ xxd -l 8 -g 1 challenge_extra.png
00000000: ff d8 ff e0 00 00 0d 0a
$ xxd -s 12 -l 4 challenge_extra.png
0000000c: 49 48 44 52  IHDR
```

# Live system — ps & netstat

| Option                               | What it does                         | Forensic value                                                 |
|--------------------------------------|--------------------------------------|----------------------------------------------------------------|
| <code>ps aux</code>                  | all processes, full detail           | complete snapshot                                              |
| <code>ps aux --sort=-%cpu</code>     | heaviest CPU first                   | catch miners and spikes                                        |
| <code>ps -o pid,ppid,user,cmd</code> | custom fields incl. parent PID       | map process lineage                                            |
| <code>ps aux --forest</code>         | tree view                            | spot injected children                                         |
| <code>netstat -tanp</code>           | TCP, all states, numeric, owning PID | no DNS lookups = fast, leaves no resolver trace; -p needs root |
| <code>netstat -l</code>              | listening sockets only               | find backdoor listeners                                        |
| <code>ss -tlnp</code>                | the modern replacement               | same idea, faster                                              |

# Pipeline companions — awk, wc, sort, uniq

| Option                             | What it does                            | Forensic value                           |
|------------------------------------|-----------------------------------------|------------------------------------------|
| <code>awk '{print \$1}'</code>     | print field 1 of each line              | extract IPs, users, methods from columns |
| <code>awk -F: '{print \$1}'</code> | custom field separator                  | /etc/passwd-style files                  |
| <code>wc -l / -c</code>            | count lines / bytes                     | sizes and totals at a glance             |
| <code>sort -rn</code>              | numeric sort, descending                | rankings from <code>uniq -c</code>       |
| <code>sort -u</code>               | sort and deduplicate                    | clean IP/URL lists                       |
| <code>uniq -c</code>               | count duplicates (input must be sorted) | frequency table of anything              |
| <code>head -n 5 / tail -n 5</code> | trim to the first/last 5                | top-N reports                            |

# awk in depth — match a field, not just a line

```
analyst@kali: ~  
  
# show lines whose FIELD 1 is the attacker:  
$ awk '$1 == "45.9.148.23"' access_big.log  
45.9.148.23 - - [26/Sep/2026:09:33:02 +0700] POST /login.php  
# exact field compare - no anchors, no escaping  
  
# the grep equivalent works, with 3 traps:  
$ grep "^45\.9\.148\.23 " access_big.log  
# ^      anchor to line start  
# \.     dots escaped (. means any char in regex)  
# space ends the field (else 45.9.148.230 matches)  
  
$ awk '$1 == "45.9.148.23" && $9 == 404' access_big.log
```

## How awk thinks

- ▶ reads line by line, splits each line into fields on spaces
- ▶ \$1 \$2 \$3 = columns, \$0 = whole line, \$NF = last column
- ▶ -F: switches the separator (passwd-style files)
- ▶ logic is built in: == != && ||, sums: s += \$5
- ▶ rule of thumb: pattern in a line -> grep; a column's value -> awk

# Navigation and reading content

```
analyst@kali: ~  
$ mkdir -p ~/forensics/lab01 && cp files/* ~/forensics/lab01/  
$ ls -la          # hidden dotfiles + permissions appear  
.bashrc .config/ challenge.png server.log update.bin  
$ find files/images \( -name "*.jpg" -o -name "*.png" \) | wc -l  
10  
  
$ cat evidence_A.txt  
CASE 1042 - EVIDENCE SNAPSHOT (5 short lines)  
$ strings /bin/bash | head -3      # readable islands in a binary sea  
/lib64/ld-linux-x86-64.so.2  
$ strings -n 8 /bin/bash | head -3 # less noise, more meaning  
$ file not_a_jpeg.jpg  
not_a_jpeg.jpg: PNG image data    # the extension is a costume
```

# The server.log case — one morning, one intruder

```
analyst@kali: ~  
$ grep -c "Failed password" server.log  
22  
$ grep "Failed password" server.log | grep -c "203.0.113.77"  
21  
# (16 against real accounts + 5 against invalid users)  
$ grep "Accepted password" server.log | grep "203.0.113.77"  
Jan 12 08:14:47 ... Accepted password for backup ...  
$ grep "shadow" server.log  
Jan 12 08:15:58 ... COMMAND=/bin/cat /etc/shadow  
Jan 12 08:15:58 ... command not allowed <- escalation DENIED  
$ grep "Too many authentication" server.log  
Jan 12 08:22:03 ... user root 203.0.113.77 ...
```

# Hashing pair and the lying header

```
analyst@kali: ~  
$ md5sum evidence_A.txt evidence_B.txt  
f690592c19051b92bc573b663f1f42c5  evidence_A.txt  
f0751c271eca176451b95da9847173f0  evidence_B.txt  
$ ls -l evidence_?.txt  
... 168 evidence_A.txt    ... 169 evidence_B.txt  
# identical to the eye - ONE trailing space apart. eyes are not verification  
  
$ file challenge_extra.png  
challenge_extra.png: JPEG image data    # <- the file LIES  
$ xxd challenge_extra.png | head -1  
00000000: ffd8 ffe0 0000 000d 4948 4452  
# IHDR at offset 12 belongs to PNG. so what is it really? (Challenge B)
```

# Full header repair — copy, inspect, patch, verify

```
analyst@kali: ~  
$ cp challenge.png challenge_A_work.png # never the original  
$ md5sum challenge_A_work.png  
84c399700e2110f5be7b84af39218126 challenge_A_work.png  
$ xxd challenge_A_work.png | head -1  
00000000: 1729 4e47 0d0a 1a0a # first 4 bytes are wrong  
# PNG wants: 89 50 4E 47 0D 0A 1A 0A  
$ hexedit challenge_A_work.png # fix 4 bytes, F2 to save  
$ file challenge_A_work.png  
challenge_A_work.png: PNG image data, 562 x 424, 8-bit/color  
$ md5sum challenge_A_work.png # changed -> hash changed  
dec3763035364d8c405fba38a5e0c58d challenge_A_work.png
```

Open the repaired image  
and read it:

**flag{d1g1tal\_f0r3ns1cs\_101}**

*upstream flag format, kept as-is*

## Why it worked

- ▶ data was intact — only the signature was broken
- ▶ 4 bytes were all that stood between data and flag

## YOUR TURN

# Now you know the moves. Solve the rest.

01

### Challenge B — 4.2

challenge\_extra.png claims to be a JPEG. You already saw the IHDR tell. Decide the true signature, repair a copy, claim the flag (ctf{xxxx}).

02

### Challenge C — 4.3

access\_big.log hides one full intrusion: find the attacker, rebuild the timeline, decode the leaked secret, and run strings on update.bin for the second flag.

03

### Working paper

staged hints in section 4, evidence numbering B-xx, hashes before and after. No evidence, no answer.